Supplement 2: Semantic Coupling & the Vocabulary Audit

Supplement 2: Semantic Coupling & the Vocabulary Audit

> *What developers name things predicts breakage better than what code calls what. Lint rules measure structure. Vocabulary measures coupling.*

The Temple Analogy

Structural coupling is the physical paved paths connecting rooms. Semantic coupling is the invisible ley lines connecting runes carved on opposite walls.

Two chambers may sit on opposite sides of the temple with no physical hallway between them. Yet if they share the same unique glowing rune, they are deeply linked. Alter the rune in Chamber A, and the energy collapses Chamber B.

In software: if a billing module and a user profile module share dense vocabulary around `AccountStatus`, they are coupled in every developer's mental model, regardless of whether they ever invoke each other's code. A subtle definition change in one silently shatters assumptions in the other.

Bavota's ICSE 2013 study proves this mathematically: developer perception of coupling tracks shared vocabulary density far more than structural dependencies flagged by static analysis tools.

The Vocabulary Audit

Run this on two supposedly decoupled services in your own codebase:

1. **Extract the nouns.** Scrape public interfaces, variable names, and comments from both services. Strip programming keywords and stop words.
2. **Calculate overlap.** Treat each collection as a set. Compute their intersection.
3. **Identify silent coupling.** A high density of shared domain terms (e.g., `OrderState`, `PaymentIntent`, `FulfillmentStatus`) without any `import` dependency between services means you have discovered a hidden ley line.

4. **Formalize the contract.** Force both teams to encode the shared vocabulary into an explicit schema. A shared protobuf definition, a JSON Schema contract, or a published type package. The format matters less than the act of making it visible and versioned.